FOCP II

AI ROAST GENERATOR

ANIMESH PASRIJA 25CSU028

PROMPT AND OUTPUT 1:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Write a simple C++ program that generates a funny roast message based on a user's name.
The program should:

Ask the user to enter their name
Store 5 roast messages
Print one roast message

Here is a simple, slightly "savage" C++ program that uses a random number generator to pick one of five roasts for the user.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Here is a simple, slightly "savage" C++ program that uses a random number generator to pick one of five roasts for the user.

C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <ctime>
#include <cstdlib>

int main() {
    // Seed the random number generator
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string name;
    std::cout << "Enter your name (if you're brave enough): ";
    std::getline(std::cin, name);

    // Store 5 roast messages in a vector
    std::vector<std::string> roasts = {
        ", I'd roast you, but my mom told me not to burn trash.",
        ", you're the reason the gene pool needs a lifeguard.",
        ", I was going to give you a nasty look, but I see you already have one.'
        ", you have a face for radio and a voice for silent movies.",
        ", if laughter is the best medicine, your face is curing the world."
    };

    // Generate a random index between 0 and 4
    int randomIndex = std::rand() % 5;

    // Print the roast
    std::cout << "\nHey " << name << roasts[randomIndex] << std::endl;

    return 0;
}
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

How it works:

- `std::getline` : We use this instead of `std::cin >>` so the program doesn't break if you enter a full name with spaces.
- `std::vector` : This acts as a container for your five roast strings.
- `std::rand() % 5` : This performs a modulo operation to ensure the random number stays within the index range of your list (0 to 4).
- `std::srand` : This "seeds" the randomizer using the current time so you don't get the exact same roast every single time you run the program.

A quick note on Logic:

In programming, we often use the modulo operator to "wrap" numbers around a range.

$$index = random_number \pmod{total_roasts}$$

This ensures that no matter how big the random number is, the result will always be a valid index for your list.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 2:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Improve the previous C++ program:

Store at least 10 roast messages in a vector
Use a placeholder {name} in each message
Replace {name} with the actual user input
Randomly select a roast using rand()
Also explain the logic before giving code.

To level up this program, we'll move away from just appending the name to the start of a sentence. By using a **placeholder**, we can make the roasts feel more natural and varied.

The Logic

1. **Placeholder Replacement**: We store our roasts with a unique tag, like `{name}`. After the user enters their name, we use the `std::string::find()` method to locate that tag and `std::string::replace()` to swap it with the actual input.
2. **Seeding the Generator**: Computers are surprisingly bad at being random. We use `std::srand(time(0))` to seed the random number generator with the current time (seconds since 1970). This ensures that every time you run the app, you likely get a different result.
3. **The Modulo Operator**: To pick a random item from the vector, we use `rand() % roasts.size()`. This takes a huge random number and "wraps" it so it falls between **0** and **9**.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

The Improved C++ Code

C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <ctime>
#include <cstdlib>

int main() {
    // Seed the random number generator with current time
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string userName;
    std::cout << "Enter your name for a personal evaluation: ";
    std::getline(std::cin, userName);

    // 10 Roast messages with the {name} placeholder
    std::vector<std::string> roasts = {
        "I'd love to insult {name}, but nature already did a better job.",
        "Some people bring joy wherever they go; {name} brings joy whenever they
        "If I had a face like {name}, I'd sue my parents.",
        "I'm not saying {name} is slow, but it takes them two hours to watch 60 
        "I would roast {name}, but I'm legally not allowed to burn garbage.",
        "I've seen more life in a 'Do Not Disturb' sign than I see in {name}.",
        "If {name} was any more useless, they'd be a 'Terms and Conditions' butto
        "Does {name} ever wonder what it's like to have a personality?",
        "I don't know what makes {name} so stupid, but it's really working.",
        "I'd agree with {name}, but then we'd both be wrong."
    };

    // Randomly select one roast
    int index = std::rand() % roasts.size();
    std::string selectedRoast = roasts[index];

    // Replace the {name} placeholder with the actual user input
    std::string target = "{name}";
    size_t pos = selectedRoast.find(target);

    if (pos != std::string::npos) {
        selectedRoast.replace(pos, target.length(), userName);
    }

    // Output the final burn
    std::cout << "\n>>> " << selectedRoast << std::endl;

    return 0;
}
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Key Improvements

- **Dynamic Sizing**: Using `roasts.size()` instead of a hardcoded number (like 10) makes it easier to add more roasts later without breaking the math.
- **String Manipulation**: The `find` and `replace` logic is a fundamental concept in C++ string handling, allowing you to treat strings like templates.
- **Encapsulation**: The roast is selected first, modified, and then printed, which is a cleaner workflow than trying to print and modify at the same time.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 3:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

The program is not taking full name input correctly.
Modify the code to:

Use getline instead of cin
Handle spaces in names properly...

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

It looks like you hit the classic C++ trap! When you use `std::cin >> name`, the computer stops reading as soon as it hits a **whitespace** (a space, tab, or newline). So, if you entered "John Doe," the variable would only capture "John," leaving "Doe" sitting in the input buffer to cause chaos later.

The Problem vs. The Solution

- **The Problem (`std::cin >>`):** Treats a space like a "stop" sign. It's great for numbers or single words, but terrible for sentences or full names.
- **The Solution (`std::getline`):** Reads the entire line of text until you hit the Enter key, allowing spaces to be part of the string.

The Fixed C++ Code

I have integrated `std::getline` and ensured the logic handles the full string correctly.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <ctime>
#include <cstdlib>

int main() {
    // Seed for true randomness
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string userName;
    std::cout << "Enter your full name (so I can insult your whole family tree):

    // FIX: Using getline instead of cin >>
    // std::cin is the stream, userName is where the data goes.
    std::getline(std::cin, userName);

    std::vector<std::string> roasts = {
        "I'd love to insult {name}, but nature already did a better job.",
        "Some people bring joy wherever they go; {name} brings joy whenever they
        "If I had a face like {name}, I'd sue my parents.",
        "I'm not saying {name} is slow, but it takes them two hours to watch 60 
        "I would roast {name}, but I'm legally not allowed to burn garbage.",
        "I've seen more life in a 'Do Not Disturb' sign than I see in {name}.",
        "If {name} was any more useless, they'd be a 'Terms and Conditions' butt
        "Does {name} ever wonder what it's like to have a personality?",
        "I don't know what makes {name} so stupid, but it's really working.",
        "I'd agree with {name}, but then we'd both be wrong."
    };

    // Randomly select a roast
    int index = std::rand() % roasts.size();
    std::string selectedRoast = roasts[index];

    // Placeholder replacement logic
    std::string target = "{name}";
    size_t pos = selectedRoast.find(target);

    if (pos != std::string::npos) {
        // This replaces "{name}" with the full string from getline
        selectedRoast.replace(pos, target.length(), userName);
    }

    std::cout << "\n>>> " << selectedRoast << std::endl;

    return 0;
}
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Why this works now:

1. `std::getline(std::cin, userName)`: This function takes the standard input stream and dumps everything—spaces included—into your `userName` string until it detects a `\n` (the newline character from your Enter key).
2. **String Integrity:** Because `userName` now holds "John Doe" instead of just "John," the `.replace()` function swaps the placeholder for the complete name.

Pro-tip: If you ever use `std::cin >>` earlier in a program before using `getline`, you might need to use `std::cin.ignore()` to clear the newline character left in the buffer, otherwise `getline` will think you hit Enter immediately and skip the input!

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 4:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

My program sometimes gives the same roast every time I run it.
Fix this issue by:

Ensuring proper randomization
Using srand(time(0))
Explain why the problem was happening and how your fix works.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

The reason your program felt like it was on "repeat" is that computers don't actually generate truly random numbers by default; they generate **pseudorandom** numbers.

The Logic: Why it was "Stuck"

The `rand()` function uses a mathematical formula to produce a sequence of numbers. By default, this formula starts with the same "seed" value every time the program launches. It's like reading the same list of "random" numbers from a book—if you always start on page one, you'll always get the same sequence.

The Fix: Seeding with Time

To fix this, we use `srand()` (seed random) combined with `time(0)`.

- `time(0)` returns the current "Unix Time" (the number of seconds elapsed since January 1, 1970).
- Since the time is constantly changing, the "seed" is different every second you run the program.
- This forces `rand()` to start at a different point in its mathematical sequence, giving you a fresh roast every time.

The Updated C++ Code

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

C++

```cpp
#include <string>
#include <vector>
#include <ctime>   // Required for time()
#include <cstdlib> // Required for rand() and srand()

int main() {
    // THE FIX: Seed the randomizer once at the very start of the program.
    // This uses the current system time as a unique starting point.
    std::srand(static_cast<unsigned int>(std::time(0)));

    std::string userName;
    std::cout << "Enter your full name: ";
    std::getline(std::cin, userName);

    std::vector<std::string> roasts = {
        "I'd love to insult {name}, but nature already did a better job.",
        "Some people bring joy wherever they go; {name} brings joy whenever they
        "If I had a face like {name}, I'd sue my parents.",
        "I'm not saying {name} is slow, but it takes them two hours to watch 60 M
        "I would roast {name}, but I'm legally not allowed to burn garbage.",
        "I've seen more life in a 'Do Not Disturb' sign than I see in {name}.",
        "If {name} was any more useless, they'd be a 'Terms and Conditions' butto
        "Does {name} ever wonder what it's like to have a personality?",
        "I don't know what makes {name} so stupid, but it's really working.",
        "I'd agree with {name}, but then we'd both be wrong."
    };

    // Now this will actually pick a different index each run
    int index = std::rand() % roasts.size();
    std::string selectedRoast = roasts[index];

    // Replace {name} placeholder
    std::string target = "{name}";
    size_t pos = selectedRoast.find(target);
    if (pos != std::string::npos) {
        selectedRoast.replace(pos, target.length(), userName);
    }

    std::cout << "\n>>> " << selectedRoast << std::endl;

    return 0;
}
```

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 5:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Improve the program further:

If the user enters an empty name, show a proper message instead of crashing
Prevent invalid input issues
Keep the code beginner-friendly and well-commented.

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

To make the program more robust, we need to add **input validation**. The biggest risk in a simple console app is the user just hitting "Enter" without typing anything, which makes the roast look broken (e.g., "I'd love to insult , but...").

The Logic for Improvements

1. **Empty String Check**: We use `.empty()` or check the `.length()` of the string. If it's empty, we use a `while` loop to repeatedly ask for a name until we get one. This prevents the "empty roast" scenario.
2. **Input Cleaning**: While `getline` is great, sometimes an invisible "newline" character from a previous input can skip the prompt. We use a validation loop to ensure the user provides actual characters.
3. **Refined Selection**: We keep the `srand` logic at the top to ensure the randomness issue stays fixed.

Why this is better for beginners:

- **The `while(true)` Loop**: This is a common pattern for input validation. It forces the user to interact properly with the program before it proceeds to the "business logic."
- **The `.empty()` Method**: This is a highly readable way to check if a string contains no characters. It's cleaner and more "C++ style" than checking if the length is zero.
- **Clear Feedback**: Instead of the program just closing or printing a blank space, the user gets a specific instruction on what they did wrong.
- **No Crashes**: By validating the input before the string replacement logic, we ensure that the `pos` search always has a valid string to work with.

Ask Gemini

Tools

Fast

Gemini is AI and can make mistakes.

PROMPT AND OUTPUT 6:

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Explain in simple terms how the roast generator program works step by step.
Also explain:

How random selection works
Why we used vector
How placeholder replacement is done

Gemini

C++ Name-Based Roast Generator

Upgrade to Google AI Plus

Think of the roast generator like a vending machine for insults. Instead of dispensing snacks, it takes your name as "payment" and drops a customized joke.

Here is the step-by-step breakdown of how the logic flows from top to bottom.

---

1. How Random Selection Works

Computers are actually very predictable, so they need a "starting point" to act random. We use two specific tools for this:

- **The Seed (`srand`):** Imagine a book filled with millions of random numbers. If you always start reading from page 1, you'll always get the same "random" numbers in the same order. `srand(time(0))` tells the computer to open the book to a page based on the exact second you pressed "Run." Since time never stops, the page is always different.
- **The Picker (`rand() % 10`):** The `rand()` function gives us a massive number (like 28,491). By using the **modulo operator (%)**, we divide that number by 10 and keep only the **remainder**. Since a remainder of a division by 10 can only be 0 through 9, we are guaranteed to get a valid index for our list.

2. Why We Used a Vector

In C++, a `std::vector` is like a smart, expandable shelf.

- **Organization:** It keeps all 10 roast strings in one container, neatly numbered (indexed) from 0 to 9.
- **Safety and Flexibility:** Unlike older "arrays," a vector knows its own size. We can ask it, "How many roasts do you have?" using `.size()`, which makes our code less likely to crash if we decide to add more roasts later.
- **Ease of Access:** It allows us to grab any roast instantly just by knowing its number, which is perfect for our random selection.

3. How Placeholder Replacement is Done

This is the "customization" phase. We don't want to just print the name at the start; we want it to fit naturally inside the sentence.

1. **The Target:** We save the text `{name}` inside our roast strings as a "bookmark."
2. **The Search:** We use the `.find()` function. The computer scans the chosen roast string character by character until it sees the first `{`.
3. **The Swap:** Once it finds the position (the index) of `{name}`, it uses the `.replace()` function. It "cuts out" the 6 characters of `{name}` and "pastes" in the user's name in that exact spot.

Ask Gemini
Tools
Fast
Gemini is AI and can make mistakes.

Gemini
C++ Name-Based Roast Generator
Upgrade to Google AI Plus

The Step-by-Step Execution Flow

Step	Action	Why?
1. Seeding	Call `srand`	Ensures you don't get the same joke twice.
2. Input	`getline`	Captures the full name, even if it has spaces.
3. Validation	`while` loop	Prevents the program from continuing if the name is empty.
4. Picking	`rand() % size`	Chooses one of the 10 stored insults.
5. Customizing	`find` & `replace`	Swaps the `{name}` tag for the actual user's name.
6. Output	`cout`	Delivers the "burn" to the console.

Export to Sheets

This combination of logic ensures the program is interactive, unpredictable, and stable!

Ask Gemini
Tools
Fast
Gemini is AI and can make mistakes.